

Hardware-in-the-loop based WCET analysis with KLEE

Marcus Lindner[†], Jorge Aparicio[†], Henrik Tjäder[†], Per Lindgren[†], and Johan Eriksson^{*}

[†]Luleå University of Technology

^{*}Grepit AB, Luleå Sweden

Email: {marcus.lindner, jorge.aparicio, henrik.tjader, per.lindgren}@ltu.se, johan@grepit.se

Abstract—C programming dominates the mainstream of embedded development as of today. To aid the development, hardware abstractions, libraries, kernels, and light-weight operating systems are commonplace. However, these typically offer little or no help to automatic worst-case execution time (WCET) estimation, and thus manual test and measurement based approaches remain the de facto standard. For this paper, we take the outset from the Real-Time For the Masses (RTFM) framework, which is developed to facilitate embedded software development for IoT devices and provides highly efficient implementations, suitable to the mainstream of embedded system design. Although the Rust language plays currently a minor part in embedded development, we believe its properties add significant improvements and thus implement our RTFM framework in Rust. We present an approach to worst-case execution time estimation in the context of RTFM tasks and critical sections, which renders sufficient information for further response time and schedulability analysis. We introduce our test bench, which utilizes the KLEE tool for automatic test vector generation and subsequently performs cycle accurate hardware-in-the-loop measurements of the generated tests. The approach is straightforward and fully automatic. Our solution bridges the gap in between measurement based and static analysis methods for WCET estimation. We demonstrate the feasibility of the approach on a running example throughout the paper and conclude with a discussion on its implications and limitations.

I. INTRODUCTION

Embedded software is playing an increasingly important role for the design and implementation of embedded devices in the CPS/IoT era. In addition to functional requirements, embedded software is often subject to requirements on robustness, safety, real-time properties, and resource efficiency.

To aid the development, software developers commonly adopt ecosystems like hardware abstraction layers, libraries, kernels, and light-weight real-time operating systems. While improving design efficiency, they typically leave the schedulability analysis at the hands of the developer. For this reason, test based methods remain the de facto standard because scheduling analysis tools are still not widely spread and typically demand expert knowledge. In comparison to static analysis approaches, target testing has the clear advantage that observations on the actual hardware obtain the results, which are free from assumptions and approximations associated to models and imprecise program analysis. On the downside however, it is a tedious and error-prone process to manually derive test vectors, instrument the code, perform tests, collect, and analyze the results. Moreover, test based scheduling

analysis of concurrent systems is an inherently hard problem and edge cases are easy to miss because testing can only cover a limited subset of task interleavings and input values. Additionally, test based approaches may also be fragile, where small changes in the system might imply large consequences to the real-time operation what in effect calls for a complete retesting of the system.

In this paper, we present a test bench for automatic worst-case execution time estimation that enables subsequent response time and schedulability analysis. We take the outset from Real-Time For the Masses (RTFM), which is a set of languages and tools being developed to facilitate embedded software development, and provide highly efficient implementations geared to static verification. The programming model of RTFM is reactive and based on the familiar notions of concurrent tasks and critical sections treated as single-unit resources. The execution model provides race-free concurrency and serialization of side effects under full control of the programmer.

The supporting Rust based RTFM implementation guarantees deadlock-free preemptive execution on bare metal single core targets by efficiently exploiting the underlying interrupt hardware for static priority scheduling and resource management under the Stack Resource Policy (SRP) [2]. Besides the advantages associated to SRP based scheduling (see Section V), various research efforts (see, e.g., the seminal work of Baker [2] and more recent publications [17], [6], [13]) established a plethora of methods to static verification (response time analysis, stack memory analysis, etc.) over the last decades.

Methods for response time and schedulability analysis typically rely on a priori WCET information. In the case of SRP based analysis, the WCETs and resource holding times for the task set are necessary to be known. Our approach takes the outset of test and measurement based methods that are well established in industry and leveraged by known methods to semi-formal program analysis [3].

The automatic test bench process comprises:

- Automated semi-formal program analysis, which derives concrete test vectors that trigger each feasible path of execution for the tasks of the system,
- Automated measurement based WCET and resource holding time estimation for the task set, and

- System wide analysis of, e.g., response times and schedulability.

In this paper, we focus on the test vector generation and automated execution time measurement and leave the system wide analysis to future work. In Section II, we review the the KLEE framework [3], the Rust programming language, and the RTFM task and resource model with its kernel implementation in Rust. In Section III, we present our approach to program analysis and test vector generation, and we outline our automated test bench for cycle accurate hardware-in-the-loop measurements of WCETs and resource holding times. Section V covers related work on WCET estimation and Section VI summarizes the contributions of the paper including an outlook to future work.

II. BACKGROUND

A. KLEE LLVM Execution Engine

KLEE [3] is a tool for symbolic execution operating on the LLVM IR (intermediate representation) of an input program. Symbolic execution is the process of executing the code through an interpreter (virtual machine) while recording path conditions and modifications to variables. In contrast to normal execution, inputs and symbolically tagged variables are assumed to be symbolic rather than having concrete values.

KLEE keeps track of the program states for each execution path and successively selects based on the chosen strategy one of the states and updates it by symbolic execution of the next instruction along the selected execution path. This process terminates when there is no further instruction to execute for any of the states or a predefined termination condition occurs. As soon as an execution path reaches its end or encounters an error (abort), KLEE discharges the path condition by means of modern solvers [19] resulting in concrete assignments to the symbolic variables, i.e., a test vector from which the exact path can be reconstructed.

According to the authors [3], “KLEE has two goals: (1) hit every line of executable code in the program and (2) detect at each dangerous operation (e.g., dereference, assertion) if any input value exists that could cause an error.” To achieve its goals, KLEE unassistedly and automatically generates high-coverage test cases. One purpose of these test cases is to detect low-level errors of generic character, i.e., errors independent of knowledge about the intended behavior of the program. Therefore, KLEE inserts branches for potentially dangerous operations to check if any input value exists that could cause an error. Such operations are pointer dereferences, zero divisions, load and store instructions for memory accesses as well as system calls. Additionally, the user can insert own assertions for which violations are discovered too. KLEE has been demonstrated to efficiently deal with complex program suites like BusyBox and core-utils [3].

As mentioned, KLEE operates on LLVM IR and is for that reason not limited to any designated input programming language. The only restriction is the availability of an LLVM compiler for the language in use. On the other hand, KLEE

provides a set of special functions to guide the symbolic execution and these functions are only available as an API for the C language. For our work, the important functions of this API are:

- `klee_make_symbolic(&v, sizeof(v), "v")`, which instructs KLEE to treat the variable `v` as symbolic and initially unknown,
- `klee_assert(c)`, which introduces a branch for the condition `c` that causes KLEE to output a test vector of concrete variable assignments for entering the false branch if for the current path condition an assertion violating value exists, and
- `klee_assume(c)`, which introduces a branch for the condition `c` that outputs an error if `c` is unsatisfiable under the current path condition (i.e., the false branch) and otherwise adds `c` to the current path condition and continues (i.e., the true branch).

B. Rust

The ambition behind Rust is to provide a memory safe systems programming language with zero-cost abstractions [18], [12], [8]. Therefore, it implements an *affine* type system with underlying memory model that guarantees memory safety and concurrency without data races. Memory safety is essentially ensured by the ownership model of Rust, which prohibits aliases for mutable data.

In Rust, each valid memory location is owned by exactly one variable. Assigning the value of a variable to another variable results in copying the value or moving the ownership. An alternative to moving the ownership is temporarily borrowing the value from a variable by referencing its memory location. Referenced values are either *immutable* (`&T`) or *mutable* (`&mut T`) and uphold the following invariant:

At any instance in time, each value `T` may be mutably referenced once or immutably referenced arbitrarily often. [22]

This invariant applies both to sequential code (`c` and `d`) and to concurrent execution contexts (`A` and `B`), as illustrated in Figure 1.

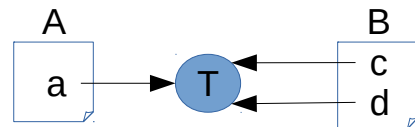


Fig. 1: Rust Memory Model

The Rust compiler enforces memory safety guarantees only in *safe* Rust code. It utilizes the *borrow checker* for static memory consistency analysis and inserts runtime verification code where static analysis is in general out of reach (e.g., array indexing). In case of failure, the runtime verification code raises an exception during execution, which is called *panic* in the Rust terminology.

Any access to shared mutable data ultimately boils down to explicitly stated *unsafe* Rust code, out of reach for the Rust

compiler to verify. Abstractions providing a *safe* API allow user code to access shared (mutable) data. Thus, given that the abstractions are sound, any program passing compilation will be memory safe by construction.

C. RTFM - Real-time for the Masses

Real-time for the masses (RTFM) is a framework intended to facilitate embedded software development of reliable real-time applications and to provide highly efficient implementations geared to static verification. RTFM aims at the needs of lightweight embedded devices and particularly real-time systems with guaranteed response time requirements such as embedded sensors and actuators. A major focus lies on providing a model of computation for concurrent real-time programming suitable for static analysis.

For the purpose of this presentation, we refer to a simplified model of computation, see, e.g., [14] for further information. A system is a set of tasks $T = \{t_1, \dots, t_n\}$ and a set of single-unit, non-preemptable resources $R = \{r_1, \dots, r_k\}$. The system specification associates a static priority $p(t_i)$, a minimum inter arrival time $ia(t_i)$, and a deadline $dl(t_i)$ to each task t_i . External and internal events trigger the tasks, which execute a finite sequence of operations with run-to-end semantics. A task t_i may claim exclusive access to a resource r_j for a named critical section $cs(t_i, r_j)$, and critical sections may only be nested in a LIFO manner. In effect, the RTFM model of computation adheres to the requirements of the Stack Resource Policy (SRP) [2] based analysis and scheduling. In this context, $\lceil r_i \rceil$ denotes the resource ceiling for the resource r_i .

Figure 2 sketches a little RTFM system composed of the three tasks (EXTI1, EXTI2, and EXTI3) and two resources (X and Y). Each task has a static priority assigned to it and the arrows indicate the resources that a task accesses. According to SRP, a resource ceiling is the maximum priority of all tasks that claim the resource.

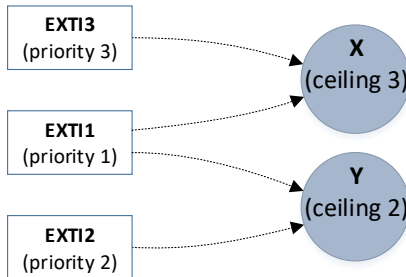


Fig. 2: The diagram exemplifies the composition of tasks and resources in an RTFM system and the derivation of the resource ceilings. Resource ceiling $\lceil X \rceil = \max(p(\text{EXTI1}), p(\text{EXTI3}))$ and $\lceil Y \rceil = \max(p(\text{EXTI1}), p(\text{EXTI2}))$, where $p(t_i)$ is the priority of task t_i .

An implementation of the RTFM framework for ARM Cortex-M microcontrollers is available as a Rust crate [10], which is the designation for a library or package in the Rust universe. The `cortex-m-rtfm` crate provides a convenient

Listing 1: `rtfm_example.rs` (abridged)

```

1 app! {
2     device: stm32f413,
3
4     resources: {
5         static X:u8 = 0;
6         static Y:u32 = 0;
7     },
8
9     tasks: {
10         EXTI1: {
11             path: exti1,
12             priority: 1,
13             resources: [X, Y],
14         },
15
16         EXTI2: {
17             path: exti2,
18             priority: 2,
19             resources: [Y],
20         },
21
22         EXTI3: {
23             path: exti3,
24             priority: 3,
25             resources: [X],
26         },
27     },
28 }

```

way to declaratively define the composition of an RTFM system and program its functionality using the Rust programming language. Subsequently, the Rust compiler yields an efficient SRP based scheduling and resource management implementation for the ARM Cortex-M families of MCUs.

Listing 1 illustrates the declarative description of the RTFM example system shown in Figure 2 encoded with the Rust crate `cortex-m-rtfm`. Lines 4-7 define the resources, which are memory resources in this case, and their respective data types. Lines 9-27 specify the tasks, their priorities, and associated resources. Furthermore, each task definition has a `path` parameter that refers to a Rust function, which is executed when the respective task is triggered by an event.

At the heart of this code snippet is line 1, which indicates that the whole RTFM system definition is processed by a Rust macro. The exclamation mark after an identifier denotes a macro in Rust. Apart from syntactic macros, Rust also provides procedural macros for general use as an experimental feature. These macros contain Rust statements that are executed at compile time during the preprocessing stage. As a result, the macro generates through its execution regular Rust code that is injected into the actual compilation stage. Natural use cases for procedural macros are syntax extensions of the programming language. The `app!` macro of the RTFM implementation is a procedural macro and can be seen as a syntax extension of the Rust language.

III. WCET ANALYSIS

In this paper, we present a method to measure the worst-case execution times (WCETs) of RTFM tasks and resource claim

blocks on the target hardware for the purpose of scheduling analysis. Our approach utilizes the KLEE tool to generate test vectors with full execution paths coverage, which are subsequently entered into the application while it is running on the target hardware. An automatic test bench measures the executing processor cycles for all test vectors at the level of tasks and resource claim blocks to determine the WCETs.

The timing diagram in Figure 3 shows a possible task interleaving when executing the RTFM example application shown in Figure 2. The lowest priority task EXT11 raises its priority according to the resource ceilings of X and Y when entering the respective claim blocks. Tasks EXT12 and EXT13 preempt the execution of lower prioritized tasks. This example demonstrates that we need to know the WCETs for all tasks and claim blocks to calculate blocking times and task interferences for a response time analysis of each task.

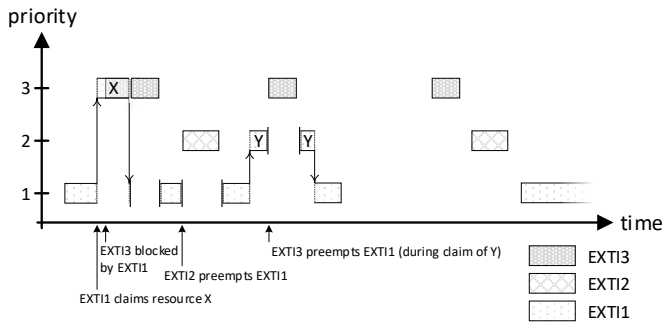


Fig. 3: The diagram shows one possible task interleaving during the execution of the RTFM example application.

For the automatic WCET measurement, our test bench is a Python script that successively:

- 1) Compiles the source code that is prepared for the execution with KLEE to LLVM IR,
- 2) Runs KLEE on the LLVM IR compiler output to generate test vectors for all execution paths,
- 3) Compiles the source code that is prepared for a controlled execution of each task separately to a binary for the target hardware,
- 4) Flashes the binary onto the target device, and
- 5) Measures the processor cycles for each task and resource claim block while iteratively executing each task with its corresponding test vectors as input.

To implement the steps 1 and 3 of our test bench, we extended the `cortex-m-rtfm` Rust crate with a `klee_mode` and a `wcet_bkpt` compiler flag for conditional compilation. Step 2 uses a custom patched version of KLEE that matches the LLVM version of the Rust compiler and runs in an isolated docker container. For control of the measurements in step 5, the Python script interfaces the GNU Debugger `gdb`.

A. Compiler flag `klee_mode`

When compiling the source code of an RTFM application with the `klee_mode` compiler flag set, the Rust compiler expands the `app!` macro to a program with a `main` function

that calls each task based on a symbolic variable and also declares the resource data to be symbolic in preparation for the execution with KLEE. Figure 4 demonstrates the principle of the `app!` macro expansion for the RTFM example application shown in Figure 2. Starting from the `main` function, each task initiates a separate execution path that declares the data of all resources associated with this task to be symbolic. This is necessary because the resource data potentially influences the execution times and due to the shared access to resources and no a priori knowledge about data constraints, we must assume each possible value to be valid.

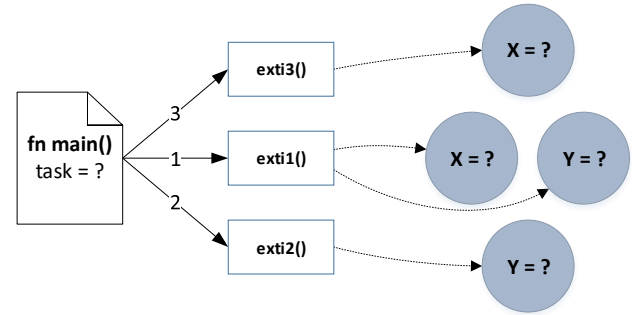


Fig. 4: The diagram visualizes the `app!` macro expansion for the RTFM example application shown in Figure 2 when the compiler flag `klee_mode` is enabled.

Listing 2 shows a simplified version of the source code from Listing 1 after the `app!` macro expansion. During the execution, KLEE finds four execution paths in the main function based on different assignments to the symbolic task variable. The last execution path (line 19) merely completes the `match` statement to satisfy the Rust compiler and is apart from that ignored. Before executing the task handlers (lines 9, 13, and 17) for the respective execution path, each resource that can be accessed during the execution is declared to be symbolic. As a result, KLEE outputs for each execution path of each task a test vector with variable assignments for all symbolic variables. After initializing the resource data with the values of a test vector, the corresponding task executes the same path again.

B. Compiler flag `wcet_bkpt`

When compiling the source code of an RTFM application with the `wcet_bkpt` compiler flag set, the Rust compiler essentially injects breakpoints at the beginning and the end of each task handler and resource claim block when it expands the `app!` macro. Our Python based test bench controls `gdb` during the measurement stage and uses these breakpoints to obtain the execution times.

After flashing the application onto the target device, the Python script iteratively takes a test vector from the output of the KLEE execution, sets the memory to the values of the test vector, initiates the corresponding task execution, and halts the task execution at the breakpoints to read the cycle counter and determine the execution times of the task and its resource claim blocks. One iteration step, i.e., one test vector, relates

Listing 2: app_macro_expansion.rs (simplified)

```

1 fn main() {
2     let mut task = 0;
3     klee_make_symbolic(&mut task, &"task\0");
4
5     match task {
6         0u32 => {
7             klee_make_symbolic(&mut X, &"X\0");
8             klee_make_symbolic(&mut Y, &"Y\0");
9             exti1(); }
10
11         1u32 => {
12             klee_make_symbolic(&mut Y, &"Y\0");
13             exti2(); }
14
15         2u32 => {
16             klee_make_symbolic(&mut X, &"X\0");
17             exti3(); }
18
19         _ => { }
20     }
21 }

```

to one execution path of a specific task. Because KLEE yields a test vector for each feasible execution path, the worst-case execution time of a task is among all the longest measured execution time for that task. The worst-case execution time for claiming a resource r_j in the context of task t_i is the longest measured execution time of all resource claim blocks $cs(t_i, r_j)$. For further discussion, see Section IV.

C. Example

Listing 3 shows three task handlers for the RTFM example application displayed in Listing 1. One can interpret this simple example as the preprocessing of a sensor input for the control output to another system.

E.g., task EXT13 is triggered by a sensor at each turn of a wheel and stores the duration of the turn in a shared resource. To get the speed of the wheel with a low error rate, EXT13 runs on the highest priority. Task EXT11 does on the lowest priority a time consuming preprocessing of the sensor reading and stores its result in another resource for task EXT12 to set a control point on request. This design makes sense when we assume that the external control system needs a prompt response when requesting input.

The Rust RTFM implementation models resource claims as a method (`claim`) that is accessible on resource instances and takes a function as an argument. The function body is the critical section for which a task claims exclusive access to the resource.

E.g., line 4 of Listing 3 illustrates how task EXT11 claims the resource X in its task handler `exti1` by calling `X.claim(...)`. The function parameter of the `claim` call is an anonymous function with access to its environment, which is called *closure* in the Rust terminology. The function or closure that represents the critical section gets access to the data of the resource through its input value. I.e., when task EXT11 claims the resource X and executes the critical

Listing 3: rtfm_example.rs (task handler)

```

1 fn exti1( ... ) {
2     let mut result: u32 = 0;
3
4     X.claim( |x| {result = *x as u32;} );
5
6     while result > 1 {
7         if result % 2 == 0 {
8             result = (result / 2) % 7;
9         } else {
10            result = (result * 3) % 7;
11        }
12    }
13
14    Y.claim( |y| {*y = result;} );
15 }
16
17
18 fn exti2( ... ) {
19     Y.claim( |y| {control_set_point(*y);} );
20 }
21
22 fn exti3( ... ) {
23     X.claim( |x| {*x = get_cycle_counter();} );
24
25     reset_cycle_counter();
26 }

```

section `result=*x as u32;`, the value of variable `x` is a reference to the memory of resource X.

Task EXT11 performs a function that reads the shared variable X and returns either 0 or 1 after a certain amount of iterations. This function has probably no useful meaning for sensor processing but it has two interesting properties to demonstrate the strength of our KLEE based WCET analysis approach. First, the amount of iterations that it takes to return the result is not equal for all input values. I.e., the time for computing the result varies with the input. Second, there is a maximum amount of iterations that it takes to return the result for an arbitrary input value. I.e., the execution time is bound and thus a worst-case execution time exists.

Even with the knowledge about these two properties, it is by no means trivial to discover the maximum number of iterations that are needed to produce a result or to find at least a sample value that triggers the maximum iterations. No testing approach can give a safe answer until all input values are tested.

When analyzing this example with our presented approach, KLEE symbolically executes all tasks and thereby explores all feasible paths. At the end of each path execution, KLEE returns a test vector of variable values that can be used to replay this specific execution path on the original program. In case of task EXT11, KLEE will discover all possible iterations for arbitrary input values and return a sample value for each instance.

Figure 5 shows the first two iteration levels of the execution paths flow chart for task EXT11. For the sake of clarity, we omit the computational steps and the branch direction. Following the left arrow of a branch means that its condition evaluates to *true* and accordingly the right side means *false*.

lead to path enumeration misses. We declare a resource to be symbolic at the entrance of a task. As a consequence, KLEE analyses the second and subsequent resource claims based on the prevailing path condition from the preceding executions. This behavior does not consider possible preemptions of higher prioritized tasks, which potentially mutate the resource data. Prohibiting multiple consecutive resource claims or forcing KLEE to recognize the resource data to be unknown each time by introducing a new resource for each claim block addresses this problem.

V. RELATED WORK

Execution time estimation has traditionally been conducted through manual test and measurement based approaches either on the actual targets or using simulated environments. However, such processes are tedious and error prone. Moreover, with growing source code complexity, it is inherently difficult to ensure that the worst-case behaviour has been exposed. To this end, academia and industry has spent much effort over the last decades to provide methods and tools that improve on manual test and measurement approaches. Ermedahl and Engblom [5] give an excellent overview to the area and layout different approaches to model based and hybrid WCET estimation. They also cover industrial experiences of commercial WCET tools, which identify a number of outstanding observations. Among these we find:

- 1) Lack of "one-click" analysis,
- 2) Lack of automatic loop bound calculation,
- 3) Structure of code has significant impact to analyzability,
- 4) Manual annotations can sometimes be burdensome to generate,
- 5) Operating-system services and interrupt handlers should be taken into account,
- 6) The large diversity in processors used in embedded systems hinders the deployment of static WCET analysis, and
- 7) Measurements and static timing analysis complement each other and give together a better understanding of the system timing and increase the trust in the resulting timing estimates.

The method presented in this paper seeks to address each observation and challenge by:

- 1) Providing a Python based test bench that fully automatically generates tests vectors with the KLEE tool and feeds them into a hardware-in-the-loop benchmark test for execution time measurement,
- 2) Tracking data dependencies throughout the complete call graph, which derives loop bounds,
- 3) Providing a well structured model of computation that is amenable to static analysis,
- 4) Preventing the need for annotations with the tracing of data dependencies and the support of conditional compilation for the analysis stage,
- 5) Unifying application code, interrupts, and operating system services with the RTFM model of computation to turn them into a part of the system wide analysis,

- 6) Performing the program analysis on the LLVM IR of the compiled source code, which opens the support for any LLVM capable target, and
- 7) Combining the advantages of static analysis and measurement in a test bench that exploits symbolic execution and hardware-in-the-loop benchmarking.

Recent related work includes [20], which concludes that creating architectural models based on the data sheets of the manufacturer is exhaustive and, while safe, may lead to ample over estimations when using the OTAWA framework.

The commercial tool RapiTime [11] is a representative example of hybrid approaches (adopting static analysis complemented by measurements). It carries out a prediction of the worst-case path and WCET by combining the obtained timing information and the structural model of the code.

Cassé et al. study overestimations due to imprecise flow facts and approximations during the analysis [4]. In comparison, our approach derives tests for all feasible paths. Hence, the derived test vectors and corresponding measurements take (up to the limitations of the KLEE tool) all traceable data dependencies into consideration and ensure to accurately expose the worst-case behaviour. Thus, our approach may improve accuracy to both architectural model based methods and hybrid techniques.

aiT ([7]) is a commercial WCET analysis package that performs static analysis on code snippets by abstract interpretation on the binary code for a wide range of target platforms. In general, aiT requires user annotations for generating meaningful output. aiT computes the upper bound including caches and pipeline interactions based on timing accurate architectural models. In general, aiT is not able to calculate the overall schedulability of the system. Instead, it lets the user characterize selected sub-parts of the system and performs a schedulability analysis using some other tool based on the derived WCETs.

SWEET ([15]) is an open source tool developed at the MRTC (Mälardalen Real-Time Research Centre). It includes a C compiler that outputs an intermediate format (ALF) used for abstract execution. This approach is quite similar to our the KLEE + LLVM approach, where KLEE operates on the LLVM IR. SWEET assumes a third party timing model that analyzes the emitted flow facts for the target architecture. Similarly to aiT, the overall schedulability analysis is out of reach for the SWEET tool.

Schedulability analysis is very important for hard real-time applications and attracts much attention in this context (see, e.g., the seminal work [16]). The theoretical work typically takes the outset of modelling the system in terms of (concurrent) tasks, which potentially support shared resources. We base our work on the Stack Resource Policy (SRP) [2]. SRP based scheduling brings a number of benefits whereof most prominent are:

- Deadlock-free execution,
- Single (non-transitive) blocking,
- Shared stack execution, and
- Single context switch per task.

In parallel to the development of the RTFM framework, Hofer et al. proposed a similar approach (SLOTH) to interrupt driven execution of OSEK models [9]. While SLOTH focuses on the scheduling and resource management of OSEK models, RTFM and its accompanying model of computation aim at providing an ecosystem for modelling, analysis, and implementation of general purpose embedded real-time software.

VI. CONCLUSION & FUTURE WORK

With this paper, we attempt to close the gap between measurement based and static analysis methods for WCET estimation. While test based methods are of practical use but can only deliver approximative results, static analysis is much more difficult to apply. Our approach utilizes the KLEE tool for a complete path exploration and subsequent hardware-in-the-loop benchmarking of the generated test input for the WCET estimations. We demonstrate the practicability of the method on the RTFM framework and discuss its implications. Although very promising, our approach has some limitations.

The classical problem of symbolic execution, i.e., path explosion, still remains. We only argue it is not an issue for lightweight embedded IoT applications, which is our target application. However, open problems are the uncertainty about the worst execution time of the generated test input for an execution path and the potential introduction of new paths after compilation to the target platform. These issues and other missing properties give space for further improvements.

WCET analysis is a precursor for response time and ultimately schedulability analysis. Our work adheres to the requirements of the Stack Resource Policy based analysis and scheduling for which a plethora of methods to static verification (response time analysis, stack memory analysis, etc.) exist. As we have seen in Section III, the proposed test bench can derive WCET estimations from which response time analysis can be readily applied and integrated into our tool for fully automatic schedulability analysis.

Currently, our approach does not model I/O. We plan to model interaction to the environment, i.e., peripheral access. The Rust based RTFM framework analogously models peripherals and resources, which allows to extend the generation of symbolic values to peripherals.

Furthermore, we plan to extend our RTFM framework to support the definition of type invariants for resources. The functions `klee_assume` and `klee_assert` for guiding the symbolic execution of KLEE allow us to establish static type invariant checks. When assuming the specified resource invariant on entering the resource claim block and asserting the invariant on leaving the claim block, KLEE verifies the validity of the invariant for all execution paths. Because resources are commonly shared among several tasks, resource invariants constitute a contract between the tasks accessing that resource and KLEE verifies the compliance. Aside from that, this extension provides another method to mitigate the path explosion problem because assumptions on the data type strengthen the path condition and hence mitigate the complexity.

REFERENCES

- [1] ARM Cortex-M4 Processor Technical Reference Manual - Revision r0p1. ARM Information Center, 100166_0001_00_en, 2015.
- [2] T. P. Baker. Stack-based scheduling for realtime processes. *Real-Time Syst.*, 3(1):67–99, Apr. 1991.
- [3] C. Cadar, D. Dunbar, and D. Engler. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, OSDI'08*, pages 209–224, Berkeley, CA, USA, 2008. USENIX Association.
- [4] H. Cassé, H. Ozaktas, and C. Rochange. A Framework to Quantify the Overestimations of Static WCET Analysis. In F. J. Cazorla, editor, *15th International Workshop on Worst-Case Execution Time Analysis (WCET 2015)*, volume 47 of *OpenAccess Series in Informatics (OASICS)*, pages 1–10, Dagstuhl, Germany, 2015. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.
- [5] A. Ermedahl and J. Engblom. Execution time analysis for embedded real-time systems. *Handbook of Real-Time Embedded Systems*, s. 35.1, 2007.
- [6] K. Hänninen, J. Mäki-Turja, M. Bohlin, J. Carlson, and M. Sjödin. Analysing stack usage in preemptive shared stack systems. Technical Report ISSN 1404-3041 ISRN MDH-MRTC-202/2006-1-SE, MDH-MRTC, July 2006.
- [7] R. Heckmann and C. Ferdinand. Worst-case execution time prediction by static program analysis. https://www.absint.com/aiT_WCET.pdf.
- [8] G. Hoare. The rust programming language. URL <http://www.rust-lang.org>, 2013.
- [9] W. Hofer, D. Lohmann, and W. Schröder-Preikschat. Sleepy Sloth: Threads as Interrupts as Threads. In L. Almeida and S. Brandt, editors, *Proceedings of the 32nd IEEE Real-Time Systems Symposium (RTSS 2011)*, pages 67–77, Los Alamitos, CA, USA, 2011.
- [10] Jorge Aparicio, Per Lindgren. cortex-m-rtfm - Real Time For the Masses (RTFM) framework for ARM Cortex-M microcontrollers, 2018. <https://crates.io/crates/cortex-m-rtfm>, last visited: 2018-04-15.
- [11] R. Kirner, P. Puschner, and I. Wenzel. Measurement-based worst-case execution time analysis using automatic test-data generation. In *IN PROC. IEEE WORKSHOP ON SOFTWARE TECH. FOR FUTURE EMBEDDED AND UBIQUITOUS SYSTS. (SEUS'05)*, pages 7–10, 2004.
- [12] S. Klabnik. *The Rust Programming Language*. No Starch Pr, City, 2017.
- [13] P. Lindgren, J. Eriksson, S. Aittamaa, P. Pietrzak, and J. Wiklander. Scheduling of CRO systems under SRP-DM. Technical report, Luleå University of Technology, 2011.
- [14] P. Lindgren, D. Pereira, J. Eriksson, M. Lindner, and L. M. Pinho. RTFM-lang Static Semantics for Systems with Mixed Criticality. In *Ada-Europe 2014: 19th International Conference on Reliable Software Technologies*, 2014.
- [15] B. Lisper. Sweet—a tool for wcet flow analysis. In *International Symposium On Leveraging Applications of Formal Methods, Verification and Validation*, pages 482–485. Springer, 2014.
- [16] C. L. Liu and J. W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *J. ACM*, 20(1):46–61, Jan. 1973.
- [17] J. Mäki-Turja and M. Nolin. Efficient implementation of tight response-times for tasks with offsets. *Real-Time Systems*, 40(1):77–116, 2008.
- [18] N. D. Matsakis and F. S. Klock II. The rust language. In *ACM SIGAda Ada Letters*, volume 34, pages 103–104. ACM, 2014.
- [19] H. Palikareva and C. Cadar. Multi-solver support in symbolic execution. In *International Conference on Computer Aided Verification (CAV 2013)*, pages 53–68, 7 2013.
- [20] M. H. Thomas Jerabek. Static worst-case execution time analysis tool development. *DEPEND 2016 The Ninth International Conference on Dependability*, 2016.
- [21] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London mathematical society*, 2(1):230–265, 1937.
- [22] Various contributors. The rust programming language, 2018. <https://doc.rust-lang.org/book/second-edition/ch04-02-references-and-borrowing.html#mutable-references>, last visited: 2018-04-15.